Lab 2: Microservices & Performance Testing

1. Overview

Data intensive applications, as the name suggests, require handling large datasets of data. This definitely requires an application that can scale. To achieve scalability, architects use microservices architecture. Microservices is an architectural style that structures an application as a collection of small autonomous services, modeled around a business domain.

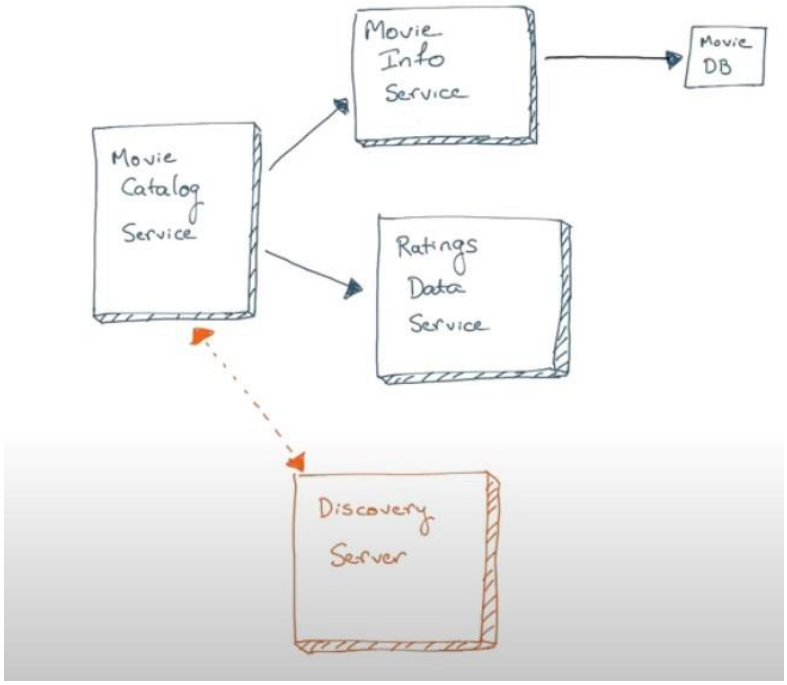
Microservices increase scalability by increasing the ability of independent deployability, which in turn allows scaling services in need only, in contrast to monolith applications. Microservices come with a great advantage of freely selecting the appropriate storage layout for each service data model.

Therefore, in this lab, you are required to experiment with a microservices application and modify its service data models. Afterwards, you will be using JMeter to performance test your application.

2. Installation

For experimenting with a ready to use microservice application, you will be using the **Movie Rating** microservices application.

Find below the github repository link:
https://github.com/yigiterinc/spring-boot-microservices
To set up the development environment, follow the instructions in the repository readme.



3. Requirements

A) Change Ratings Service Storage Data Model to MySQL

It is required to change the Ratings service storage data model from the default in-memory to MySQL. To do that, your entry point is the `RatingsResource` class. There is a static list being returned there. You will have to fix that.

This means that you are required to:

1. Install MySQL.
2. Design a schema to hold the ratings data.
3. Modify the class to use MySQL when querying for ratings.

Why did we recommend a relational DB (My SQL) for this service? Think of use cases where this choice is both adequate or inappropriate in terms of operations latency and throughput.

B) Cache the MovieDB query results in MongoDB

The provided implementation fetches movie info from **the movie DB** service API. You are required to cache this result in MongoDB to avoid such calls. You should show the effect of adding caching on the performance of the system for the endpoints involved in calling this service. You might need to generate data to test (see section D) these endpoints.

Note: You can modify the code to fetch data from the movie DB or your source of data (could be generated on the fly). Be careful, your generated data should be retrieved after some delay similar to that of the movie DB to avoid misleading results. You should at least performance test the endpoint as it returns 10 million movies in response to the invocation.

This means that you are required to :

1. Install MongoDB.
2. Design a schema to hold the cached results.
3. Modify the `MovieResource` class to cache results in MongoDB.
4. Modify the `MovieResource` class to fetch from MongoDB first and fall back to querying **the movie db** otherwise.
5. Test the architecture before and after adding the cache layer by showing the average latency, P90 latency, and throughput of the endpoint **http://localhost:8082/movies/{movieId}**.

Why did we suggest caching in this service, not other services? Try to suggest caching always where caching might matter.

C) Create a new Trending Movies Service with gRPC API

Create a new service that can be queried by the catalog service for finding out top 10 movies depending on ratings.

This means that you are required to:

1. Create a new folder for the new service. (***Hint: start by copying any of the service's folders and edit it.)***
2. Implement gRPC API for answering the query: **what are the top 10 movies by rating?**
3. Add a new API that calls trending movies service from the catalog service using gRPC
4. Test the new catalog service API by querying the top 10 movies.

From which data source does this service fetch its data? Is this data source adequate for this service use case of it? Is there a method to improve the performance of the use case in this service?

D) Run JMeter Tests

Now it is time to test our application. Here we will use JMeter to do a performance test and stress test for our application.

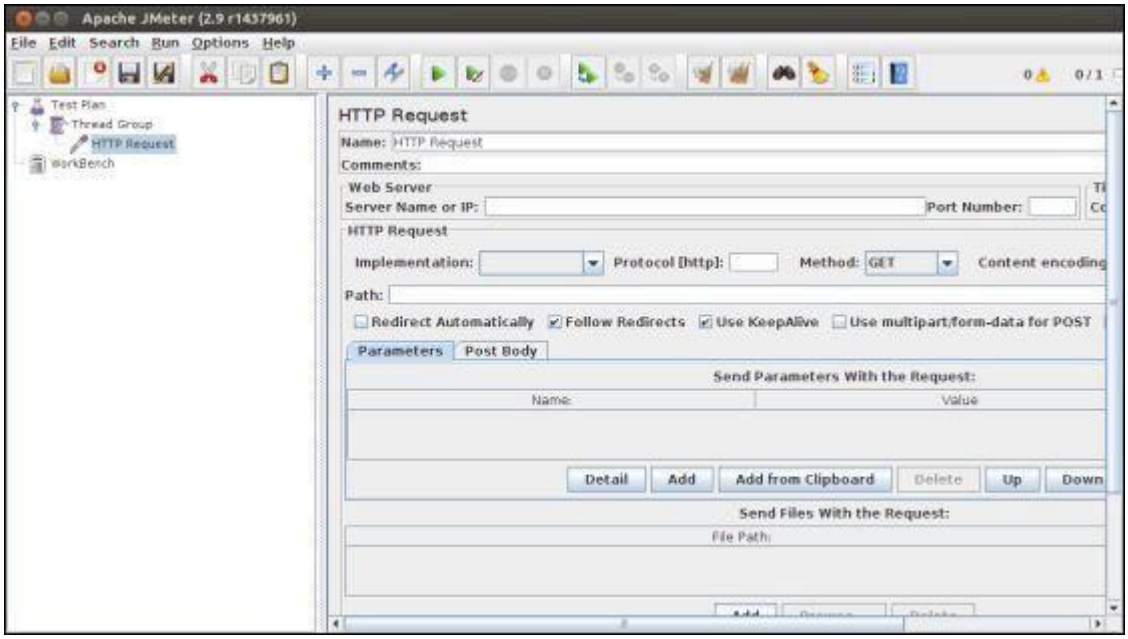
Performance Test – This test sets the best possible performance expectation under a given configuration of infrastructure. It also highlights early in the testing process if any changes need to be made before the application goes into production.

Stress Test – This test is an attempt to break the system by overwhelming its resources.

(Recall: comparison between performance and stress testing in Lecture 01)

To achieve that:

1. Download JMeter from this link https://jmeter.apache.org/download_jmeter.cgi.
2. After extracting the downloaded file, go to the bin folder, and start JMeter.
3. The JMeter window will show up a “Test Plan” and rename it to “Performance Test”.
4. Add a Thread Group element to your test Plan, use it to configure the following:
    a. Setting the number of threads
    b. Setting the ramp-up time
    c. Setting the number of test iterations
5. Add a HTTP Request sampler to the thread group. The following screen shows what you should see at this step.



6. Fill in the parameters of the Web Server (e.g., localhost) and set the port value. Then at the “Path” section put the URL for the WebService that you want to call.
7. Run the test plan, and see the reports.

8. Save the test plan to a new file named “PerformanceTest”, and repeat the same steps again for the “StressTest”. **<u>The details of the test plans are left to you as an exercise</u>**

5. Deliverables

You are required to deliver the following:

- MySQL schema
- MongoDB schema
- Proof of caching benefit (median and P90 Latency and throughput before and after)
- A running version of the new application
- JMeter Test Plans (jmx files for both performance and stress tests)
- JMeter measurements in any chart you see fit (line, column, etc,) and conclusion which includes:
- Latency distribution under normal load
- Throughput
- Maximum number of requests that your server fulfill without failure - Answers to all discussion questions - Report containing all of the above.

6. Notes

- You are allowed to alter the data in the databases to test the top 10 movies scenarios.
- You should work in groups of **four**
- All team members should be ready to answer any question during discussion ● **Any cheating will be severely penalized.**